

FINDING THE AVAILABILITY WEBSITE NAME BY USING MACHINE LEARNING

Kanokphon Kane

School of Information Technology Mae Fah Luang University
Chiang Rai, Thailand 57100
6031501001@lamduan.mfu.ac.th

Raiwin Jantawong

School of Information Technology Mae Fah Luang University
Chiang Rai, Thailand 57100
6031501048@lamduan.mfu.ac.th

Abstract—This study aims to predict good text or dirty text on all web. The property consists of the URL of the website and result pass and block. We have to compare the concept of classification with the Naïve Bayes classifier and how to create a decision tree in the classification of independency property data by comparing the ideal models for discrete data (Naïve Bayes Classifier). And the model that can handle continuous data efficiently (K-Nearest Neighbor Classifier). The study found the percentage accuracy in classification as Naive Bayes Classifier (99.93 percent) is less than accuracy in the K-Nearest Neighbor Classifier was 99.93 cause of training data set contains boundary data volume making models of classification for K-Nearest Neighbor Classifier is not suitable for testing data. Because they have to share the data with an attribute that has a value frequency of a word or quantitative data. And concluded that gender, age, and grade were not affected and disabilities of learning. The attribute that affects specific learning disabilities, including the total scores in reading, the total rate of writing, and the appearance or behavior in terms of normal intelligence quotient (IQ), respectively, to the appropriate classification of the test data. The researchers have suggested that they should experiment with different models suitable for quantitative data and analyze the relationship between the properties of the data to improve the initial decision. As a result of this study is only a preliminary screening to assume only not exactly diagnose children with disabilities in learning variable data from samples in the schools. If the information from the medical support, you can refer to in practice.

Keyword- Machine Learning, Naive Bayes, Security.
KNN ,Dataset

I. INTRODUCTION

Nowadays, AI is part of science computers that are about designing intelligent computing systems. AI can demonstrate human behavior in the area of understanding language, learning, reasoning, or solving problems. And from the past to the present, technology has advanced a lot. Humans then create a system that can make computers. Can think and make decisions with the least flaws. It is programmed into the computer to think, reason, and learn more by itself. Here it is used to solve complex math problems using Naive Bayes, which are part of Machine Learning and as a Subset of Artificial Intelligent (AI). Which the classification this will be categorized in the form of Supervised Learning, used to separate text, that is, The project will have to train the model and then use the model. The use of Naive Bayes, which is a predictive method using the Classifier theory of probability, that the term is the desired word, block, or pass for example, Tease then suck is the word you want to block. In the application of Naive Bayes, The project use text grouping, document classification, so that the machine can sort the group by itself. After the model is obtained, it will be inserted into the firewall to allow the security system to block or pass access to the domain name from external data. Such as the use of internet networks in various organizations

Therefore, this system will create an AI that can separate words from text using model Naive Bayes to predict the

results. Then train the model and put it in the firewall to be able to block or pass access to the domain name from external data.

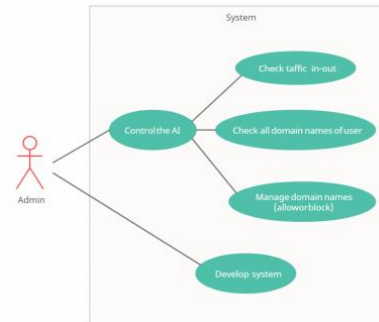


Fig. 1 Use case diagram.

II. KEY TECHNOLOGY

A. Network Security

First, Network Security is vital in protecting client data and information, keeping shared data secure and ensuring reliable access and network performance as well as protection from cyber threats. A well-designed network security solution reduces overhead expenses and safeguards organizations from costly losses that occur from a data breach or other security incident. Ensuring legitimate access to systems, applications and data enables business operations and delivery of services and products to users. Every good network security system uses a combination of different types of network security tools to create a layered defense system. The theory behind this strategy is that if a threat manages to slip past one security countermeasure, the other layers will prevent it from gaining entry to the network. Each layer provides active monitoring, identification, and threat remediation capabilities in order to keep the network as secure as possible.

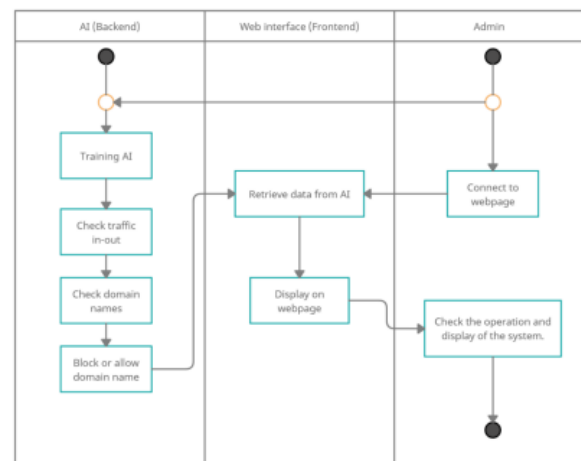


Fig. 2 Diagram proposed system architecture.

B. Naive Bayes Classifier

Naive Bayes is an algorithm which is commonly used in natural language processing (NLP) tasks such as spam filtering, sentiment analysis, classification, and recommendation. It is based on Bayes' Theorem as shown below.

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Fig. 3 Naive Bayes Classifier.

$P(A|B)$ = probability of A if B occurs

$P(B|A)$ = probability of B if A occurs

$P(A)$ = probability of A

$P(B)$ = probability of B

A Naive Bayes classifier gives us the conditional probabilities of events occurring related to each other by using Bayes' Theorem. If you would like to go deep into it, please visit here. The project will use Multinomial in this notebook.

Multinomial: Naive Bayes, it works with frequencies of features. It is used text classification or

Basics of Natural Language Processing

Since The project will use Naive Bayes in a text classification task, I would like to briefly explain some concepts that we are going to use.

- Lemmatization: It is a process of making the same words in their stem. For example, run, ran, running is different in terms of Python. The project lemmatizes the word to get run.
- Stop words: The words that are not important in terms of the context
- Tokenization: The process of extracting words in a sentence by spaces and punctuations. In this project we'll use nltk's word tokenize.

III. METHODS

The project will explain the step of working of the system by the first step will take about working of the web interface. And following by working of machine learning that used model of naive bayes including predicting the result.

A. Abbreviations and AcData Collection

The data used in this research are various texts obtained from social media such as Facebook, Twitter, Pornhub and other dirty websites. The data was labeled into two categories, not safe and safe. Not safe is the class containing hate speech or offensive language while safe is a class that does not contain both. The total amount of the data is 8887 with data for the safe category and 195200 for the not safe category.

B. Text Preprocessing

Raw data which has been obtained from the previous stage are unstructured text and still contains a lot of noise. Text preprocessing will be applied to these raw data so that the data are ready for use. Text preprocessing includes removing tags and URIs, removing punctuation, stop word removal, and

URIs Removal: In this stage, HTML tags and URIs were removed because the research only focuses on the words and category.

Stop word Removal: Stop word is a common word that often appears in a sentence but has no meaning. The stop word list is taken from English stop word.

Get value: get is the process of getting word or text to machine learning.

C. Web interface

A web interface is a web page that displays the output or output. The work starts with the URL of the website we want to find the result. Then when the scan button is pressed The web page will order Python to work. Or that is to order the model to work by the web page to send a URL that is inserted into the model, after which the model will read URL acquired and make a prediction of the results. It will then return to the web page. Showing the result that the website blocked or passed.

D. Python Filter

Python filters are used to filter text from URLs on request. The get command is to read a web page from a URL. It reads from the website's HTML and receives the html code like this to get the website's html code. Usually, a web page contains html code that defines the structure and rendering style of the entire web. If you open a web page and right click on an empty place and select view source, you will be able to see the same code. The get command reads the web and returns the result as a Response-type object, which stores the information on the web page. If you look at the text attribute you will get the text in html decoded as text. We will use the text obtained in the test model. With code as follows.

```
URL = 'Insert URL of Wab'
res = requests.get(url)
html_page = res.content
soup = BeautifulSoup(html_page, 'html.parser')
text = soup.find_all(text=True)
output = ""
blacklist=['document','noscript','header','html','meta','head',
'input','script',]
for t in text:
    if t.parent.name not in blacklist:
        output += '{ } '.format(t)
texts=(soup.get_text())
```

E. Machine Learning: Naive Bayes Classifier

Naive Bayes is part of Machine Learning and is a subset of Artificial Intelligent (AI). Classification This will be categorized in a supervised learning model, that is, the project will need to teach the model information and then implement the model. The project will use Naive Bayes to predict the results and have the following steps:

1. Prepare Data

Input the data from csv and then make it a DataFrame. The project will only extract field text and use CountVectorizer to tokenize and start wrapping. The project has all the dataset 201361 row.
df = pd.read_csv("Book2.csv")
all_list=DataFrame(df)

```
display(all_list)
arr_text = all_list.iloc[:, 'text'].values
vect = CountVectorizer(stop_words='english',
lowercase=True)
count_train = vect.fit(arr_text)
Followed by making a transform into One Hot Encoding
format.
Transformer = vect.transform(arr_text)
```

Prepare tf-idf. Start with Config tf-idf and show the idf value. TF-IDF is a keyword-based extraction technique by weighting each word by using two factors: TF and IDF. TF stands for Term Frequency or term frequency. The purpose of TF is to find out how often each word appears in each document. Finding the TF value is found by the formula: $TF(t, d) = \text{number of words appearing in document} / \text{Total number of words in document}$.

All The project has to do is to find the TF value in every document, here is 201359 documents. Next, find the IDF value. IDF stands for Inverse Document Frequency or its inversion in document frequency. Which is derived from documents that have a lot of TF values, still does not answer that it is an important word because TF will focus on each document, but IDF is the value calculated from all documents. Can be General to some extent, the IDF value can be calculated from:

$$IDF(t) = \log((1 + n) / (1 + DF(t))) + 1$$

After that the project multiply the TF and IDF values so that the weight of the TF and the weight of the IDF, which are opposites, are weights that can separate the keyword. `tfidf_transformer = TfidfTransformer(smooth_idf=False).fit(transformer)`

2. Train Data

After making the statement, The project will train by first preparing the class in an array like `arr_text`, then train from the transformer. When the train is complete, The project will create a model using `multinomialNB` and create a test. Here The project will use array `text[0]` for testing.

```
detect_model = MultinomialNB().fit(messages_tfidf, arr_class)
arr_text[0]
```

Then convert it to a vector and store the value in one interpreter to predict that the prediction will come out as a Block that matches Expected. Bringing all the data to the train to test it should be matched (Expected is 100%).

```
print('Predicted: ', detect_model.predict(messages_tfidf)[0])
print('Expected: ', arr_class[0])
```

```
Predicted: Block
Expected: Block
```

Fig. 4. Test prediction form train data.

3. Prediction's result

Once the model is trained, the project will try the test with a new word. The new word the project will add will be the URL of the web that has filtered the text by using the python filter to filter text.

For example, the web URL `www.google.com` The result is Pass. And if it is `www.pornhub.org` the result is Block.

```
url = 'http://www.google.com'
res = requests.get(url)
html_page = res.content
soup = BeautifulSoup(html_page, 'html.parser')
text = soup.find_all(text=True)
output = ''
blacklist = ['document', 'noscript', 'header', 'html', 'meta', 'head', 'input', 'script',]
for t in text:
    if t.parent.name not in blacklist:
        output += '{} '.format(t)
texts=(soup.get_text())
datatext = re.sub(r'\n', '', texts)

testtext = datatext
vectmp = vect.transform([testtext])
tftmp = tfidf_transformer.transform(vectmp)
result=detect_model.predict(tftmp)
resulti=''.join(result)
print(resulti)

Pass
```

Fig. 5. Website result Pass.

```
url = 'http://www.pornhub.org'
res = requests.get(url)
html_page = res.content
soup = BeautifulSoup(html_page, 'html.parser')
text = soup.find_all(text=True)
output = ''
blacklist = ['document', 'noscript', 'header', 'html', 'meta', 'head', 'input', 'script',]
for t in text:
    if t.parent.name not in blacklist:
        output += '{} '.format(t)
texts=(soup.get_text())
datatext = re.sub(r'\n', '', texts)

testtext = datatext
vectmp = vect.transform([testtext])
tftmp = tfidf_transformer.transform(vectmp)
result=detect_model.predict(tftmp)
resulti=''.join(result)
print(resulti)

Block
```

Fig. 6. Website result Block.

F. Network security

A firewall is a network security device that monitors incoming and outgoing network traffic and permits or blocks data packets based on a set of security rules. Its purpose is to establish a barrier between your internal network and incoming traffic from external sources (such as the internet) in order to block malicious traffic like viruses and hackers.

Firewalls carefully analyze incoming traffic based on pre-established rules, the rules here are that Machine Learning is built. and filter traffic coming from unsecured or suspicious sources to prevent attacks. Firewalls guard traffic at a computer's entry point called ports, which is where information is exchanged with external devices.

IV. RESULT AND DISCUSSION

A. Web interface

In this design, the project builds a user-friendly interface for the admin to manage and check the URL of the website. In this design, Admin can, add, and delete URL safety and unsafety. Users can only URL safely because they are safe from malware. The Figure showed the user interface in this design.



Fig. 7. Admin Web Page.

B. Functionality test

Functionality test is to make sure that the functionality of URI from web, action, and response from server work as expected. We measure the performance of the server from the response of the server when the client requests service by its response message. The response message contains:

- Pass: Informational It means the request was received and the process is artificial intelligence prediction URL of the website is safe to use for user and admin
- Block: Informational It means the request was received and the process is artificial intelligence prediction URL of the website is unsafe and maybe have malware in URL of the website to use for user and admin.

Table below shows the conclusion of the response from the server side following with its status code and response message.

TABLE I. THE FUNCTIONALITY TEST TABLE

Name (URI)	Method	Result
https://www.netflix.com/search?q=sex&jbv=81245964	REQUEST/GET	Pass
https://www.netflix.com/search?q=sex&jbv=81179107	REQUEST/GET	Pass
https://en.wikipedia.org/wiki/Sex	REQUEST/GET	Pass
https://www.empflix.com/amateur-porn/miss-web-teen/video2676741	REQUEST/GET	Block
https://www.deadgoodbooks.co.uk/new-crime-tv-shows-2020/	REQUEST/GET	Pass
https://dirtyship.com/lena-the-plug-fucking-facial-cumshot-porn-video-leaked/	REQUEST/GET	Block
https://www.porntube.com/?cid=2930	REQUEST/GET	Block

https://www1.javbuz.com/movie/hito-mi-serizawa--2979.html	REQUEST/GET	Block
https://www.binance.com/th	REQUEST/GET	Pass
https://xhamster.one/?ref=pornduke	REQUEST/GET	Block

	precision	recall	f1-score	support
Block	0.98	1.00	0.99	187414
Pass	0.98	0.71	0.82	13945
accuracy			0.98	201359
macro avg	0.98	0.85	0.91	201359
weighted avg	0.98	0.98	0.98	201359

Accuracy: 0.9787543640959679

Fig. 8. Accuracy of Naive Bayes Classifier

V. CONCLUSION AND DISCUSSION

In this paper, different classification techniques such as Naive Bayes, KNN, were used to evaluate the accuracy for predicting URL Websites. The experiment was conducted on the dataset to python filter before preparing to ready predicting text. Different accuracy measures like precision, recall, and f1-score were also used to compare their effectiveness. The results reveal that Naïve Bayes is best among all the classifiers that have been applied for URL of website prediction and introducing machine learning helps network security, increases user safety, and can monitor the performance of the network. The model is a Naive Bayes classification that can be used to really improve performance and can check URL of web site safety or unsafe for benefit users.

Experimenting with this dataset with other models to compare the K-Nearest Neighbor (KNN) and Decision tree, it was found that both models had high accuracy 99.93 which is more than Naive Bayes. but the train was difficult to train. It is not suitable for Text data and during a long time to train, for example, KNN is suitable for numerical data. The project has to convert it to numbers before the project can train the data. To do this job, KNN compares words that are divided into important attributes with the total number of datasets, thus taking a very long time to train. Decision tree, the project needs to find the optimal max-depth, it takes a very long time to do this work because there are so many datasets used, so it is not suitable for this model. Therefore, the project think Naive Bayes is the perfect model for this dataset because it is well suited for both text and large-data applications. It works faster than other models. This is the reason for the Naive Bayes Classification.

ACKNOWLEDGMENT

This work would not have been successful without the kind support of the advisor, committee, and all support

personnel. The project would like to thank everyone for their advice and help. Thanks to Aj.Mahama Sebakor, Advisor of senior project who supported the project within this study. And thanks to the School of information technology, computer engineering, Mae Fah Luang University for supporting this research.

REFERENCES

- [1] A. McCallum and K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification," Working Notes of the AAAI/ICML Workshop on Learning for Text, Palo Alto, CA: AAAI Press, 1998, pp. 41–48. I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.
- [2] Z. Yun-tao, G. Ling and W. Yong-cheng, "An improved TF-IDF approach for text classification", *J. Zhejiang Univ. Sci*, vol. 6A, no. 1, pp. 49-55, 2005.
- [3] K. Ming Leung, Naive bayesian classifier, Polytechnic University Department of Computer Science/Finance and Risk Engineering, 2007.
- [4] F. Yang, "Eliciting an Overlooked Aspect of Bayesian Reasoning", *ACM SIGCSE Bulletin*, vol. 39, no. 4, pp. 45-48, 2007
- [5] V. Krammer, "Privacy Security and Trust", PST '08. Sixth Annual Conference on An Effective Defense against Intrusive Web Advertising, pp. 3-14, 2008.
- [6] Tiliang Zhang, Hua Zhang and Fei Gao, "A Malicious Advertising Detection Scheme Based on the Depth of URL Strategy", 2013 Sixth International Symposium on Computational Intelligence and Design (ISCID), vol. 2, pp. 57-60, 2013.
- [7] S. Shalev-Shwartz and S. Ben-david, *Understanding Machine Learning: From Theory to Algorithms*, New York, 2014.
- [8] P. Tsangaratos and I. Ilia, "Comparison of a logistic regression and Naïve Bayes classifier in landslide susceptibility assessments: The influence of models complexity and training dataset size", *Catena*, vol. 145, pp. 164-179, 2016.
- [9] P.N. Tan, M. Steinbach and V. Kumar, *Introduction to Data Mining*, Pearson/Addison Wesley, 2006.
- [10] T.K. Wu, S.C. Huang and YR. Meng, "IdentiJYing and Diagnosing Students with Learning Disabilities," *International Joint Conference on Neural Networks*, 2006, pp 4387-4394.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove template text from your paper may result in your paper not being published.

We suggest that you use a text box to insert a graphic (which is ideally a 300 dpi TIFF or EPS file, with all fonts embedded) because, in an MSW document, this method is somewhat more stable than directly inserting a picture.

To have non-visible rules on your frame, use the MSWord "Format" pull-down menu, select Text Box > Colors and Lines to choose No Fill and No Line.